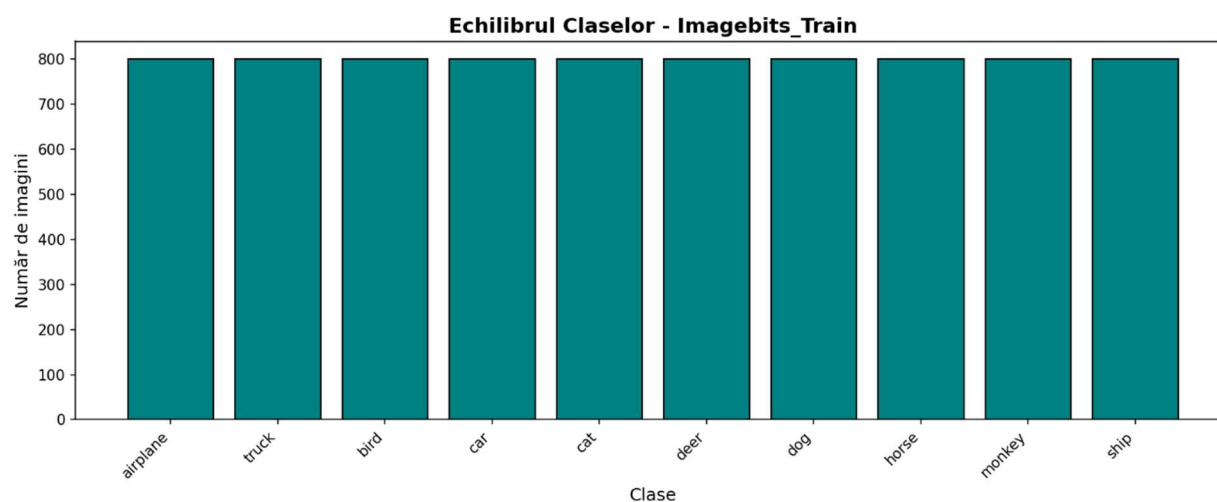
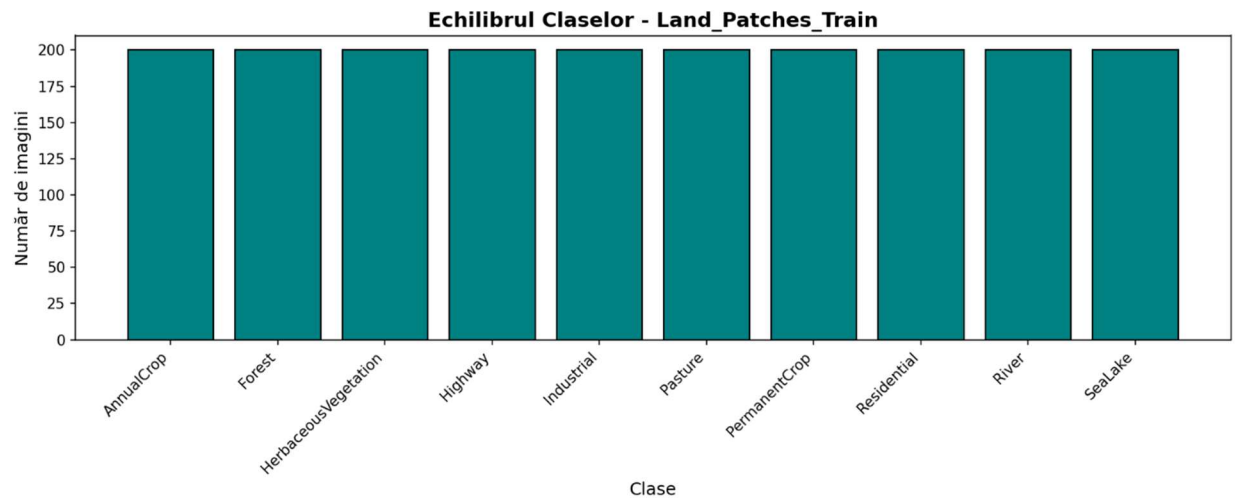
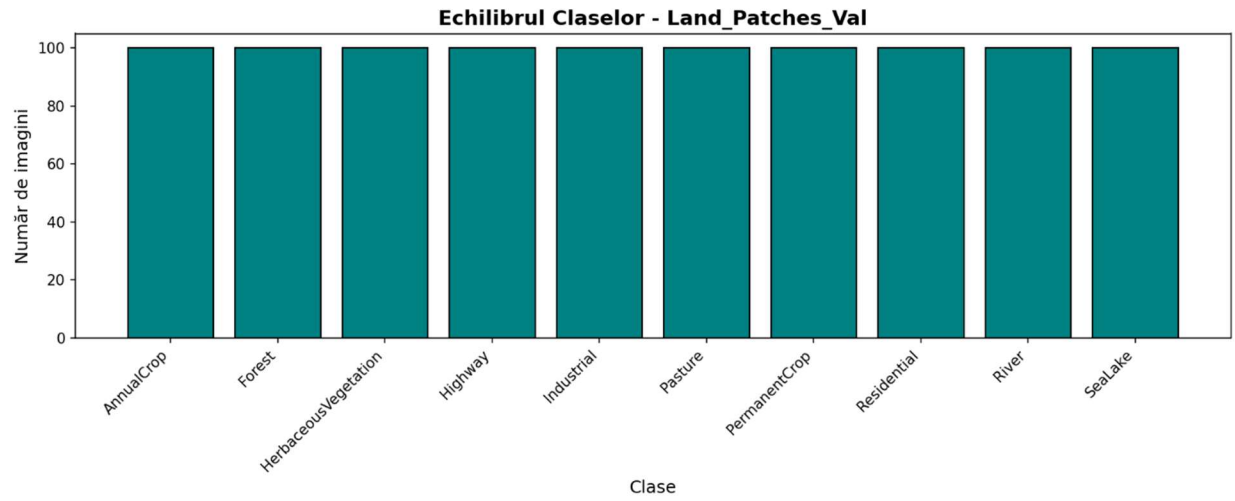


Raport Tema 2 Invatare Automata – Goanță Rareș-Ștefan

I. Imagebits si Landpatches

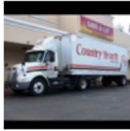
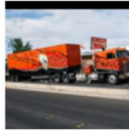
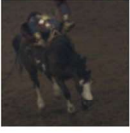
Explorarea Datelor:



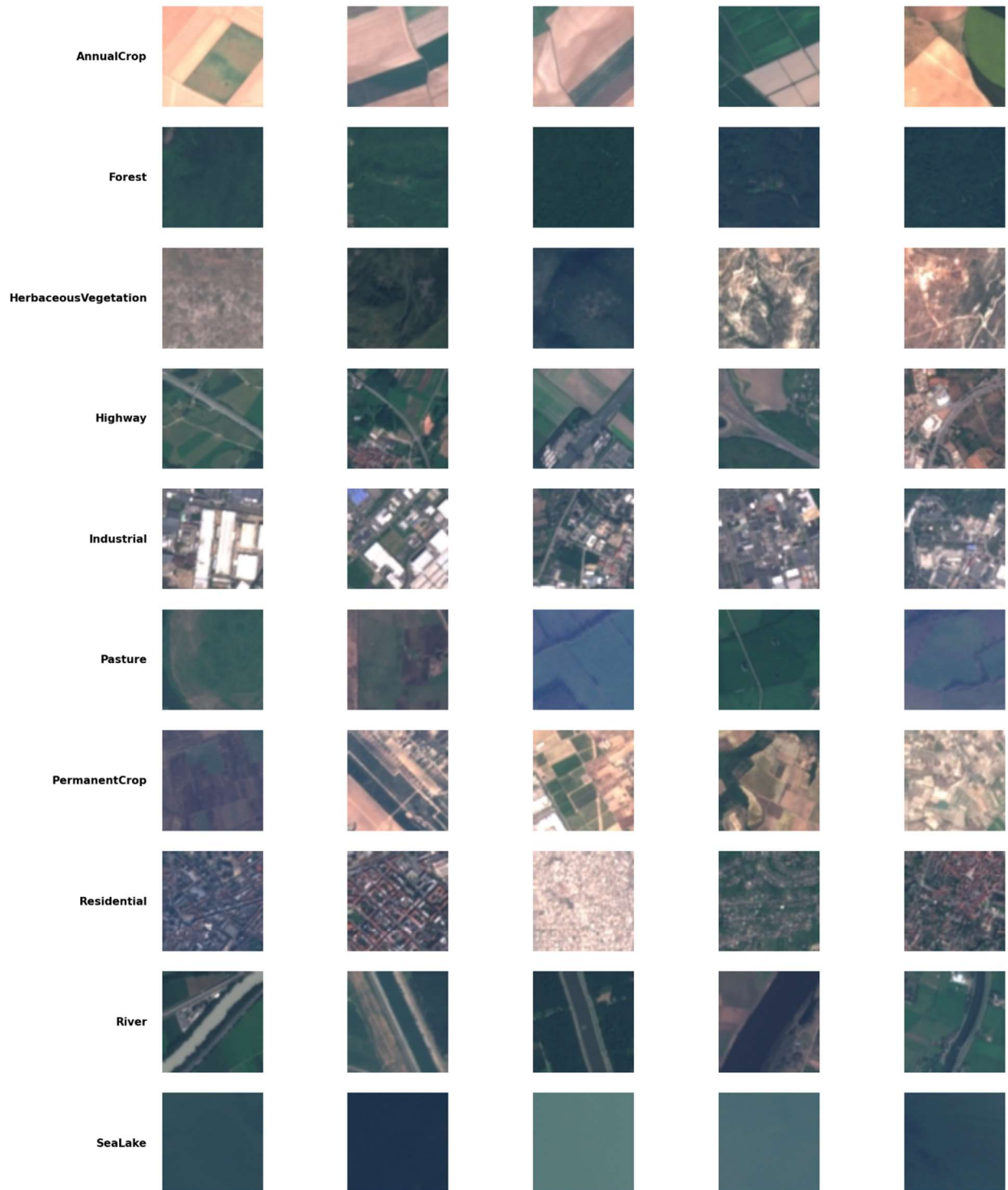


Datele sunt echilibrate si suficiente.

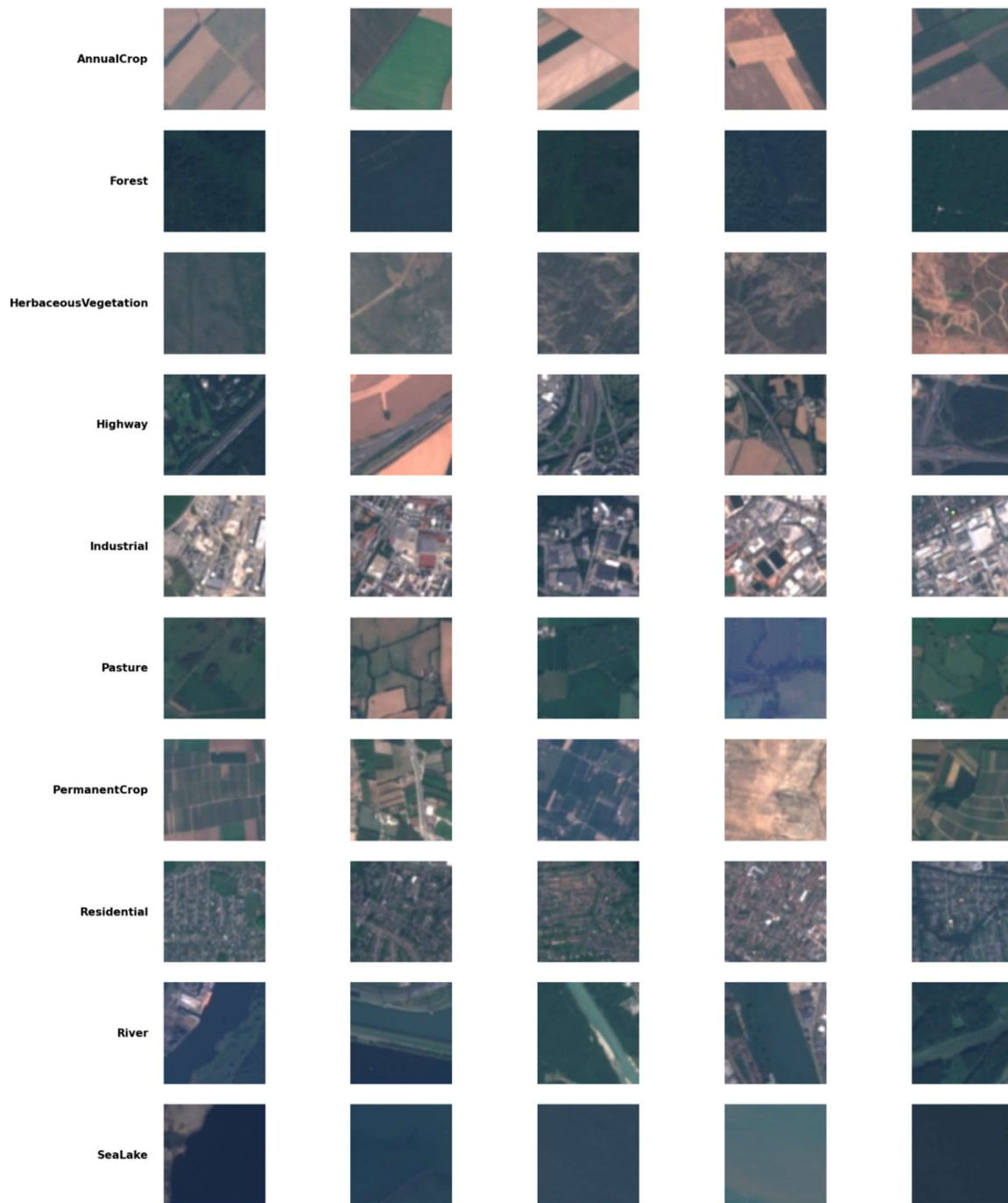
Variabilitate intra-clasă - Imagebits_Train

airplane					
truck					
bird					
car					
cat					
deer					
dog					
horse					
monkey					
ship					

Variabilitate intra-clasă - Land_Patches_Train



Variabilitate intra-clasă - Land_Patches_Val



Se observa variatiile intre clase si intra-clasa. Unele sunt mai usor de detectat(ex:sealake) decat altele si exista similaritati intre clase(ex:pasture si permanentcrop).

Am observant ca Imagebits are rezolutie nativa 96x96, in timp ce Landpatches are 64x64, asa ca le-am adus la rezolutia comuna de 96x96 pentru a putea folosi aceleasi modele pe ambele. Nu am ales 64x64 pentru a nu pierde informatie din cele din Imagebits.

MLP:

- Arhitectura:

- Flatten: $3 \times 96 \times 96 = 27,648 \rightarrow 512$

- BatchNorm1d + ReLU + Dropout(0.3)

- 512 $\rightarrow$ 256

- BatchNorm1d + ReLU + Dropout(0.3)

- 256 $\rightarrow$ num_classes

- Justificari:

 - BatchNorm1d: stabilizează antrenarea, permite learning rate mai mare

 - Dropout(0.3): previne overfitting pe MLP care tinde să memorize

 - 2 straturi ascunse: echilibru între capacitate și risc overfitting

- Hyperparametri si alegeri de implementare:

- Initial:

 - Batch size : 64

 - Epoci : 15

 - LR : 0.001

 - Optimizer : Adam (weight_decay=1e-4)

 - Loss : CrossEntropyLoss (standard)

- Dupa prima incercare, modelul facea overfit foarte mult si dadea rezultate slabe

- Am marit numarul de epoci si am adaugat early stopping

- La landpatches am plecat de la cel mai bun model antrenat pe Imagebits si am pus $LR=LR/10$ deoarece pornim de la un model cu greutati bune deja, deci schimbarile sunt mai fine

- Am adaugat :

 - Learning Rate Scheduling

- weight_decay=1e_3 – regularizare mai agreziva -> mai putin overfitting
- augmentari suplimentare
- criterion = CrossEntropyLoss(label_smoothing=0.1)
- Mariri performanta:
 - Imagebits:
 - Fara augmentari: 44.88% -> 47.48%
 - Cu augmentari: 47.72% -> 53.42%
 - Landpatches:
 - 57.26% -> 61.23%

CNN:

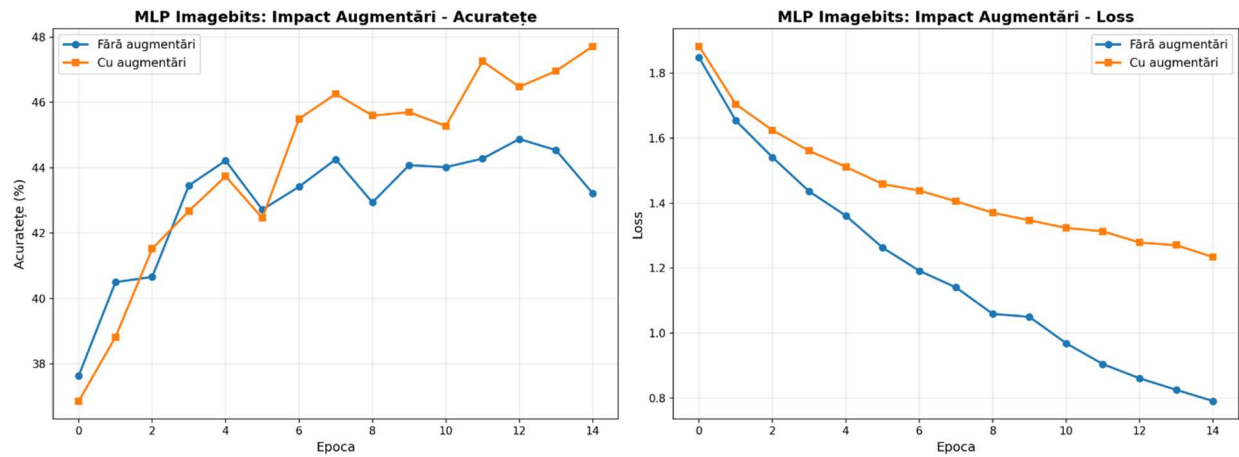
- Arhitectura:

- Inspiratie LeNet
- Conv2d(3→32, 5×5) + BatchNorm2d + ReLU + MaxPool2d(2) -> 48×48
- Conv2d(32→64, 5×5) + BatchNorm2d + ReLU + MaxPool2d(2) + Dropout2d(0.25) -> 24×24
- Conv2d(64→128, 3×3) + BatchNorm2d + ReLU + MaxPool2d(2) → 12×12
- AdaptiveAvgPool2d(1×1) → 128
- Linear(128 → 10)

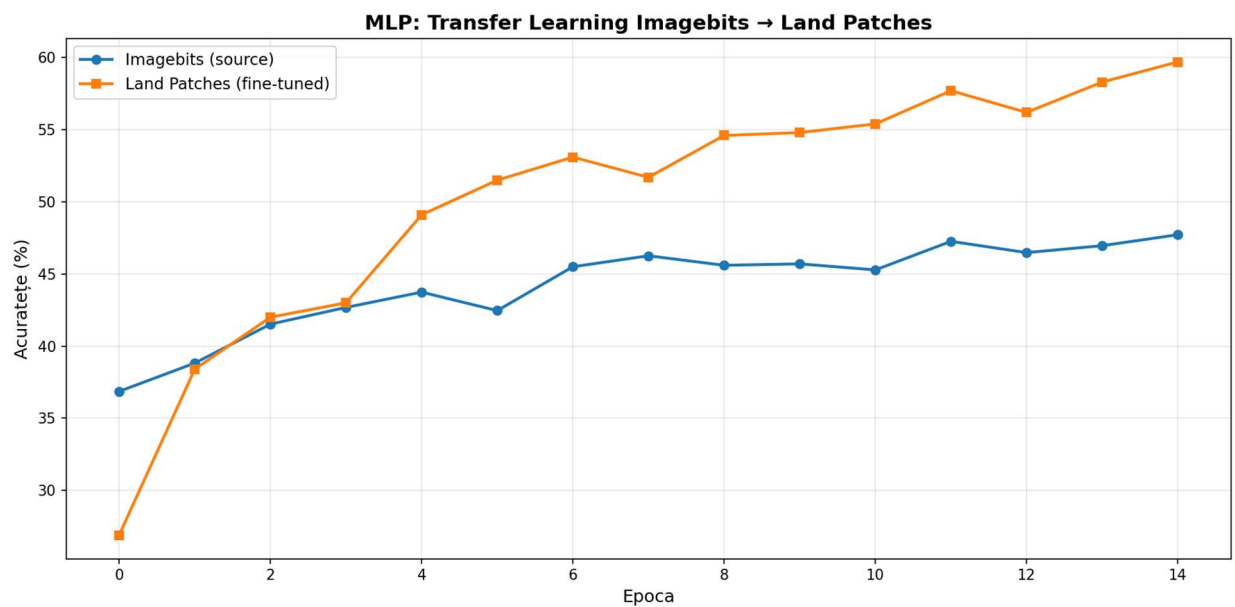
- Aceiasi hyperparametri si aproximativ aceleasi modificari(augmentari diferite)

- Mariri performanta:
 - Imagebits:
 - Fara augmentari: 57.38% -> 70.36%
 - Cu augmentari: 56.44% -> 65.90%
 - Landpatches:
 - 68.54% -> 84.96%

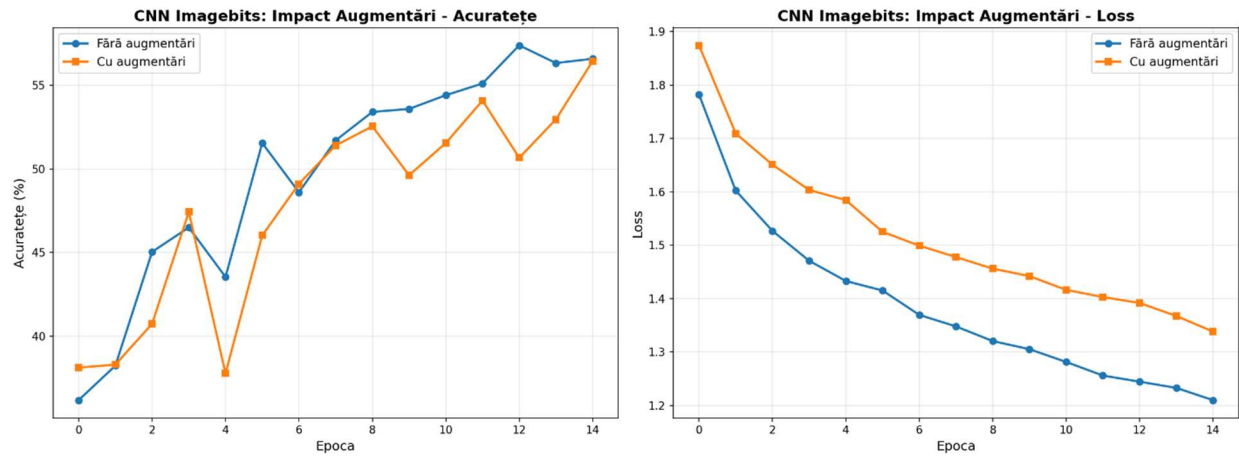
Versiunea initiala:



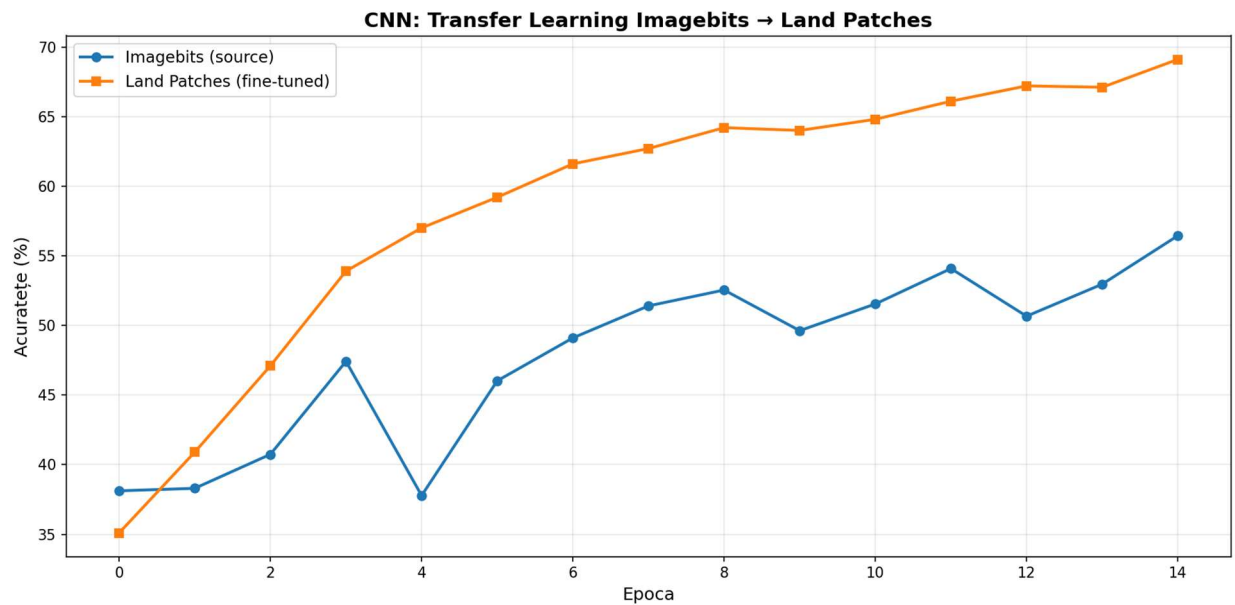
Se observa ca augmentarile ajuta la MLP.



Se observa convergenta mai rapida prin transfer learning.

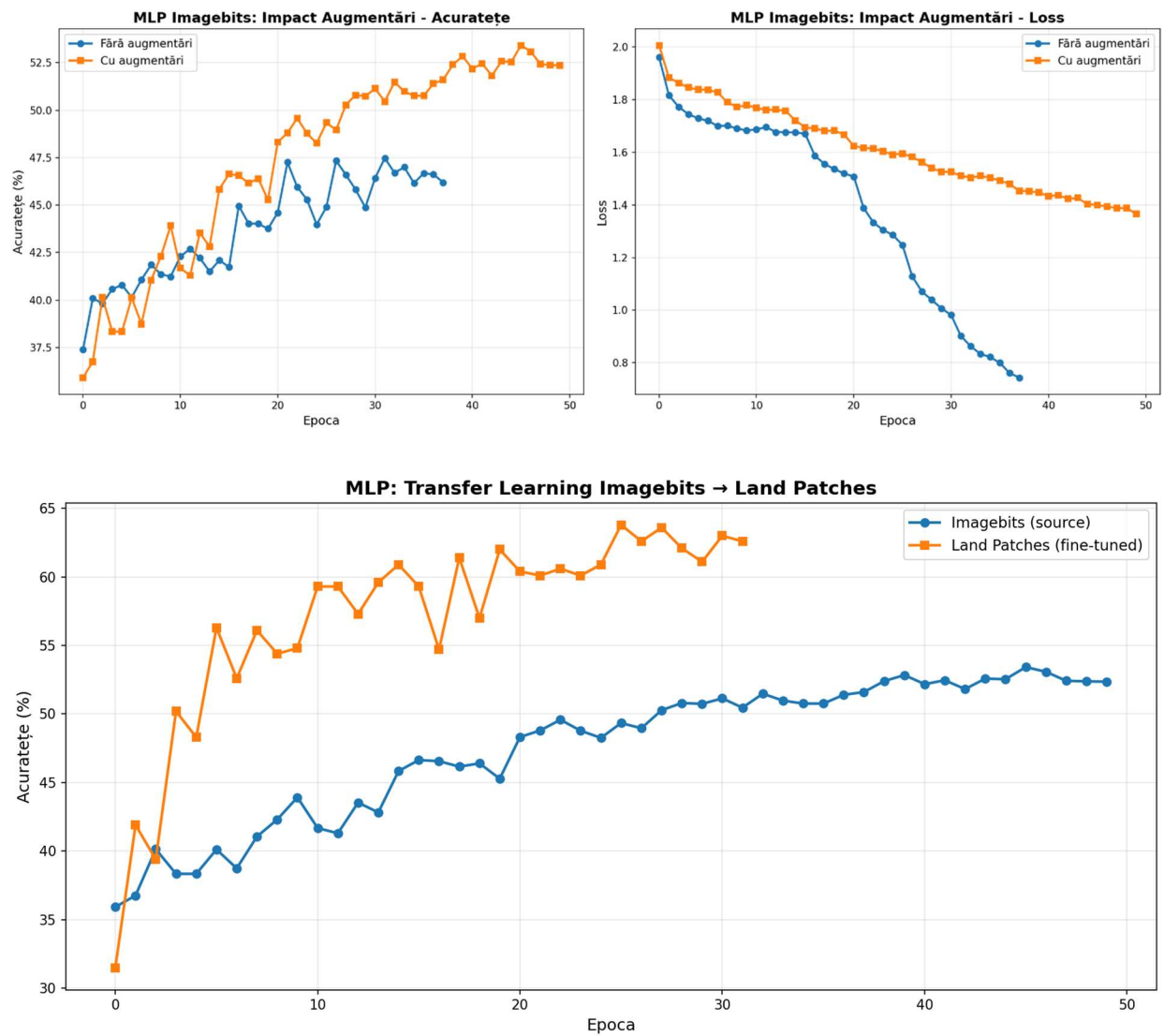


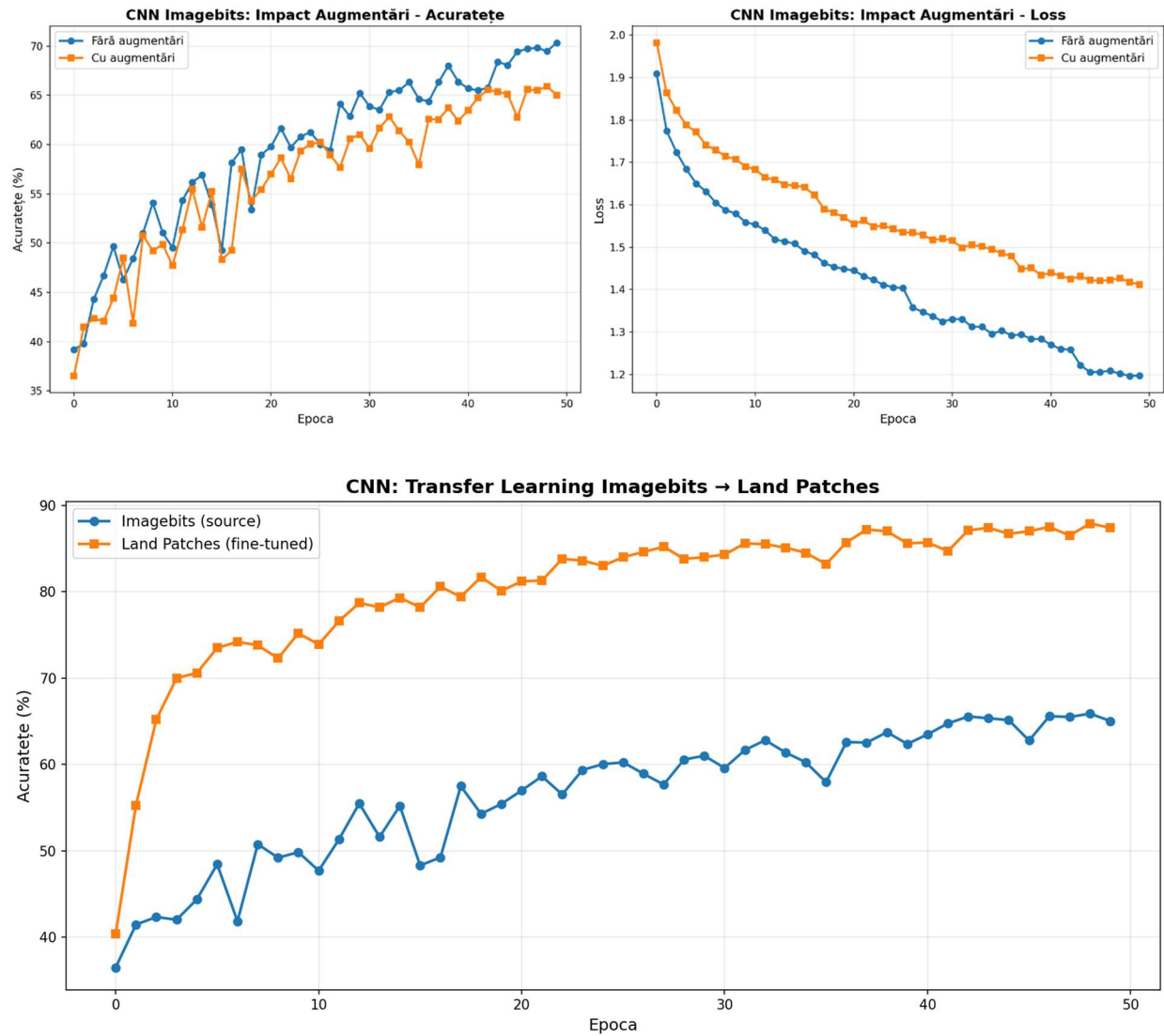
Augmentările în plus chiar scad din acuratența la CNN. Prea multe augmentări au distrus informația relevantă.



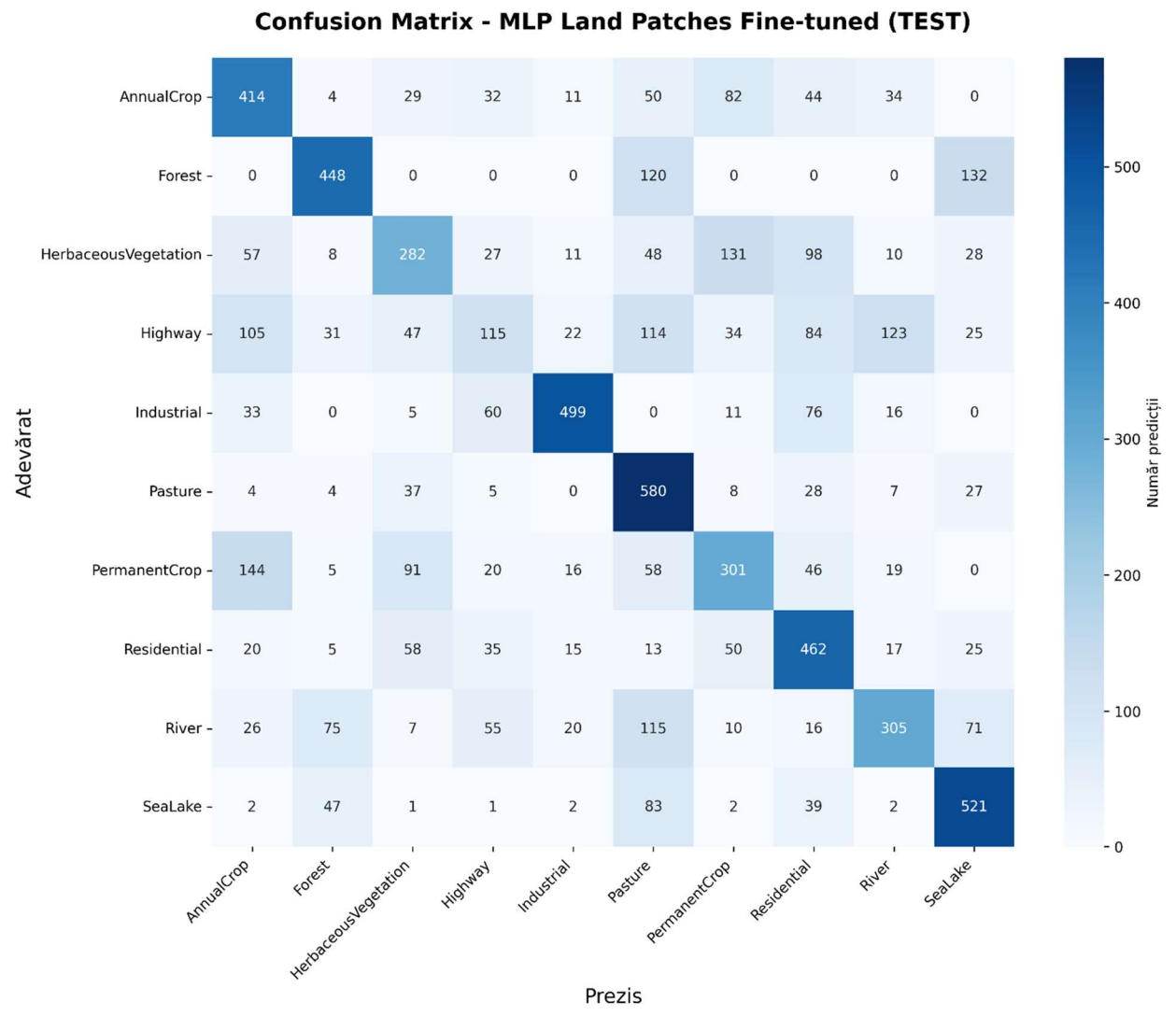
Aceeași observație ca la MLP.

Verisunea finala:

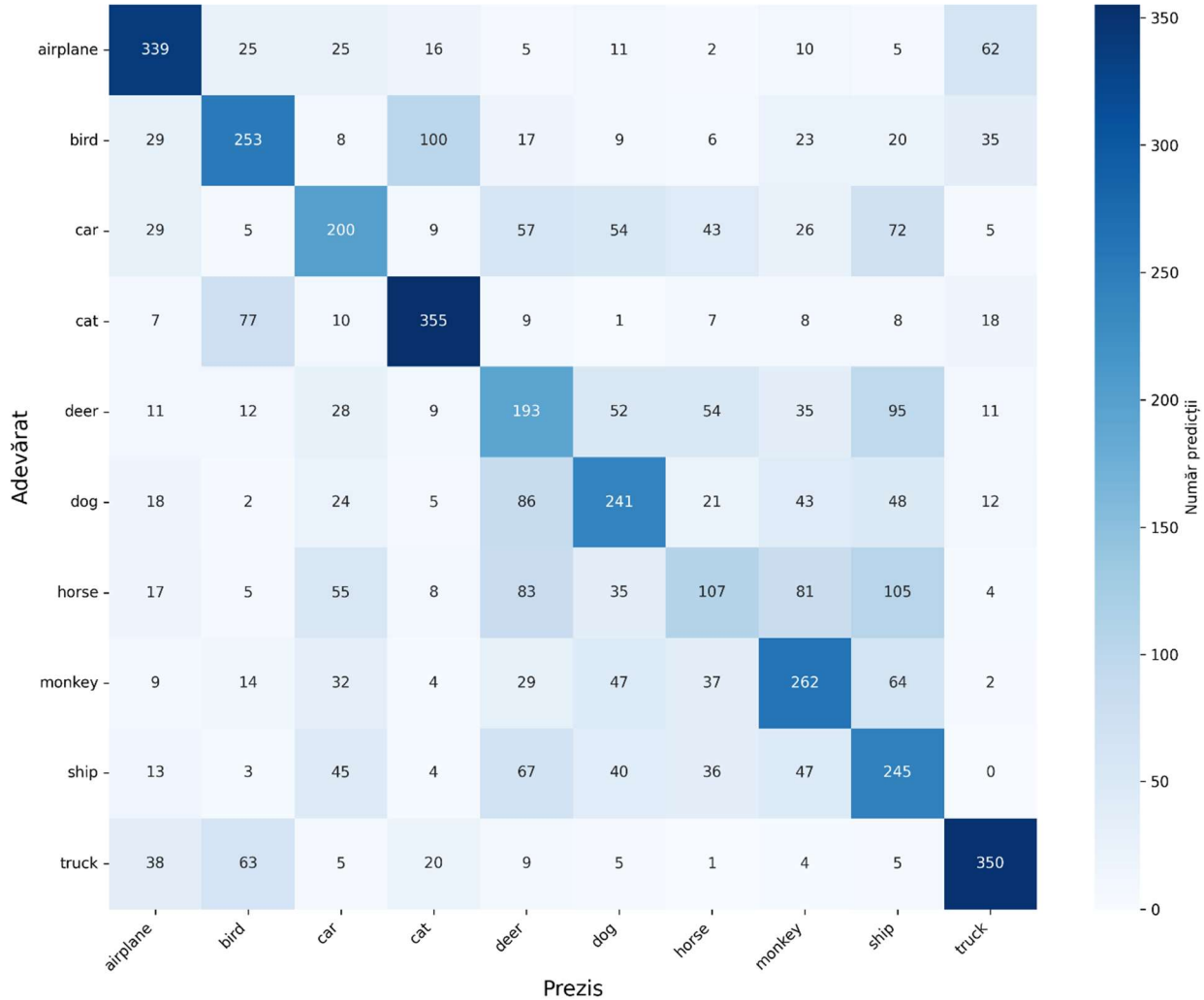




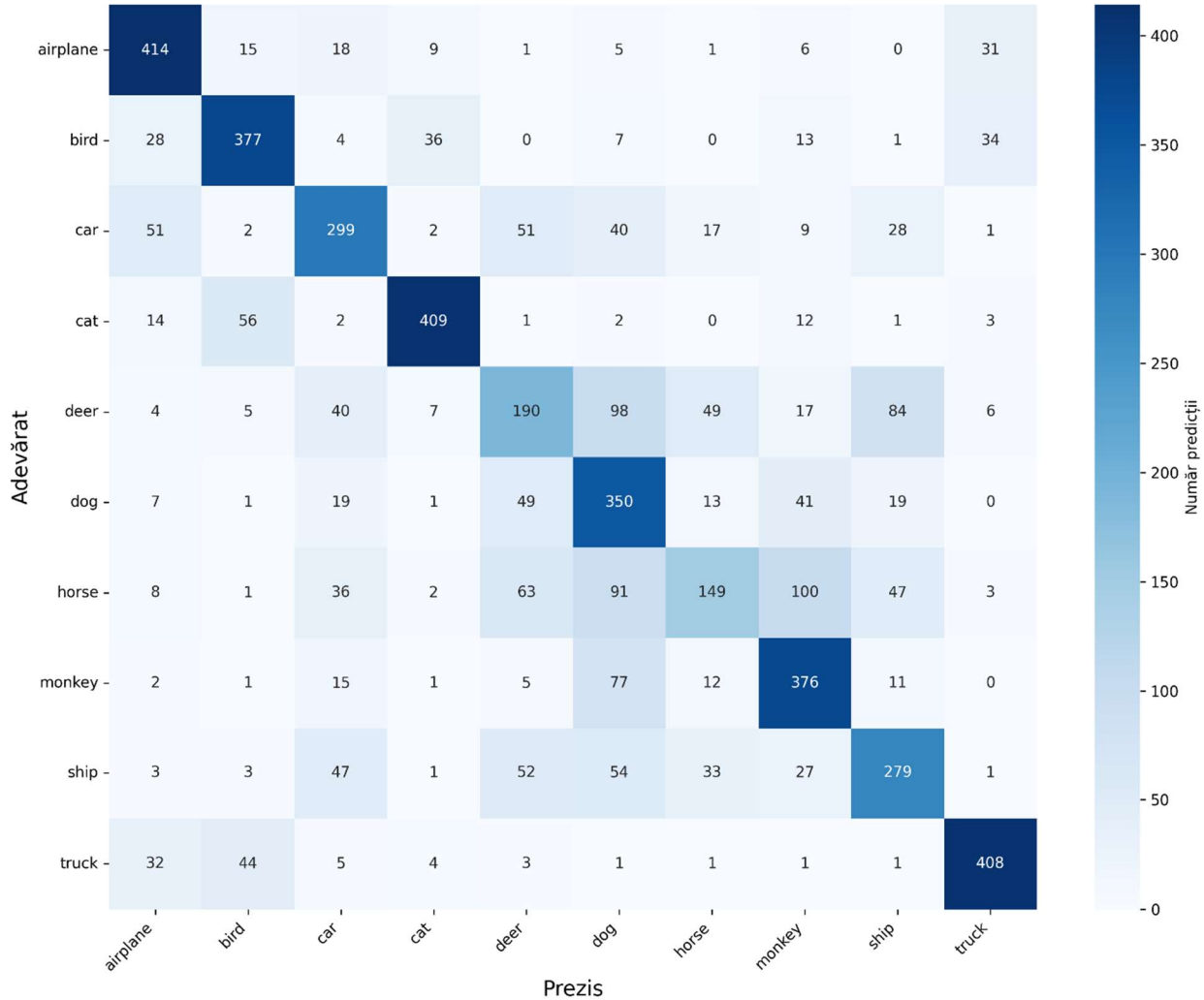
Observatiile sunt aceleasi ca la prima varianta, insa rezultatele si diferentele sunt mai evidente/mari.



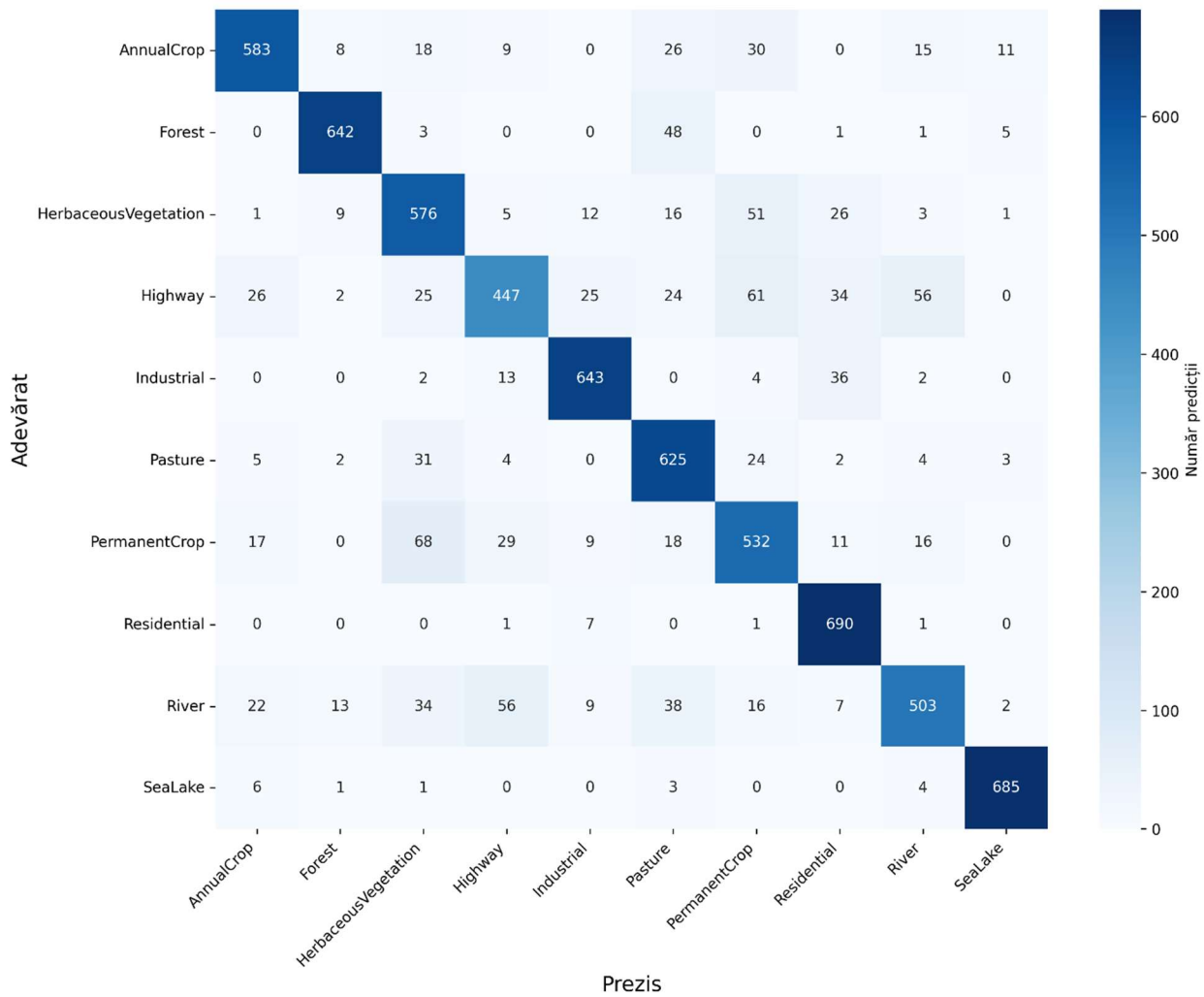
Confusion Matrix - MLP Imagebits (cu augmentări)



Confusion Matrix - CNN Imagebits (cu augmentări)



Confusion Matrix - CNN Land Patches Fine-tuned (TEST)



=====
 Confusion Matrix - MLP Imagebits (cu augmentări)
 =====

	precision	recall	f1-score	support
airplane	0.6647	0.6780	0.6713	500
bird	0.5512	0.5060	0.5276	500
car	0.4630	0.4000	0.4292	500
cat	0.6698	0.7100	0.6893	500
deer	0.3477	0.3860	0.3659	500
dog	0.4869	0.4820	0.4844	500
horse	0.3408	0.2140	0.2629	500
monkey	0.4861	0.5240	0.5043	500
ship	0.3673	0.4900	0.4199	500
truck	0.7014	0.7000	0.7007	500
accuracy			0.5090	5000
macro avg	0.5079	0.5090	0.5056	5000
weighted avg	0.5079	0.5090	0.5056	5000

=====
 Confusion Matrix - CNN Imagebits (cu augmentări)
 =====

	precision	recall	f1-score	support
airplane	0.7353	0.8280	0.7789	500
bird	0.7465	0.7540	0.7502	500
car	0.6165	0.5980	0.6071	500
cat	0.8665	0.8180	0.8416	500
deer	0.4578	0.3800	0.4153	500
dog	0.4828	0.7000	0.5714	500
horse	0.5418	0.2980	0.3845	500
monkey	0.6246	0.7520	0.6824	500
ship	0.5924	0.5580	0.5747	500
truck	0.8378	0.8160	0.8267	500
accuracy			0.6502	5000
macro avg	0.6502	0.6502	0.6433	5000
weighted avg	0.6502	0.6502	0.6433	5000

=====
 Confusion Matrix - MLP Land Patches Fine-tuned (TEST)
 =====

	precision	recall	f1-score	support
AnnualCrop	0.5143	0.5914	0.5502	700
Forest	0.7145	0.6400	0.6752	700
HerbaceousVegetation	0.5063	0.4029	0.4487	700
Highway	0.3286	0.1643	0.2190	700
Industrial	0.8372	0.7129	0.7701	700
Pasture	0.4911	0.8286	0.6167	700
PermanentCrop	0.4785	0.4300	0.4530	700
Residential	0.5174	0.6600	0.5800	700

River	0.5722	0.4357	0.4947	700
SeaLake	0.6285	0.7443	0.6815	700
accuracy			0.5610	7000
macro avg	0.5589	0.5610	0.5489	7000
weighted avg	0.5589	0.5610	0.5489	7000

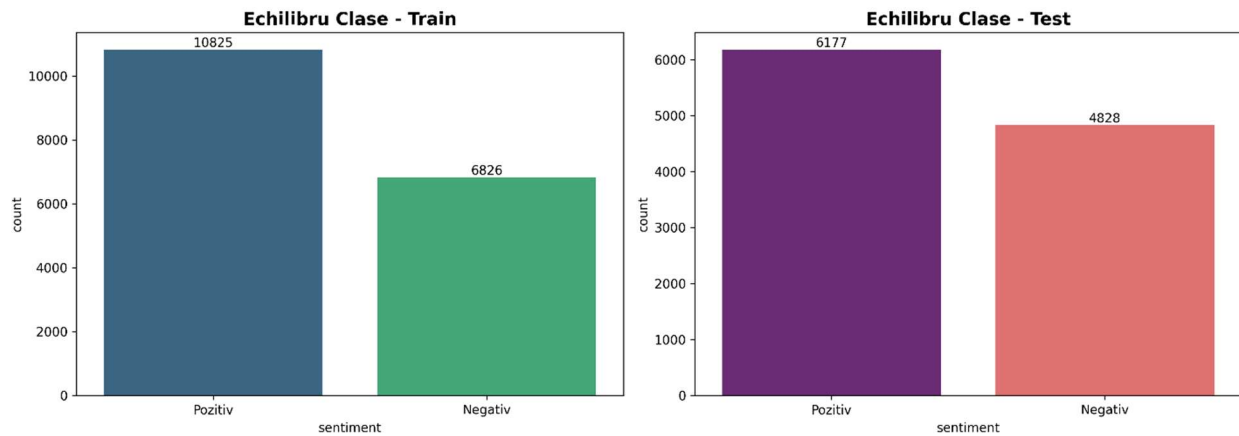
=====
 Confusion Matrix - CNN Land Patches Fine-tuned (TEST)
 =====

	precision	recall	f1-score	support
AnnualCrop	0.8833	0.8329	0.8574	700
Forest	0.9483	0.9171	0.9325	700
HerbaceousVegetation	0.7599	0.8229	0.7901	700
Highway	0.7926	0.6386	0.7073	700
Industrial	0.9121	0.9186	0.9153	700
Pasture	0.7832	0.8929	0.8344	700
PermanentCrop	0.7399	0.7600	0.7498	700
Residential	0.8550	0.9857	0.9157	700

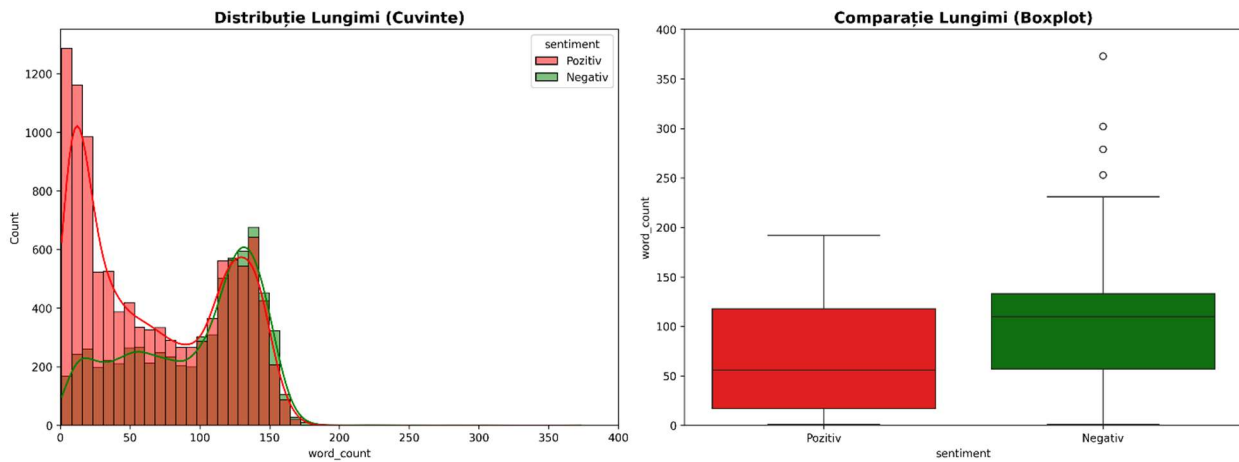
River	0.8314	0.7186	0.7709	700
SeaLake	0.9689	0.9786	0.9737	700
accuracy			0.8466	7000
macro avg	0.8475	0.8466	0.8447	7000
weighted avg	0.8475	0.8466	0.8447	7000

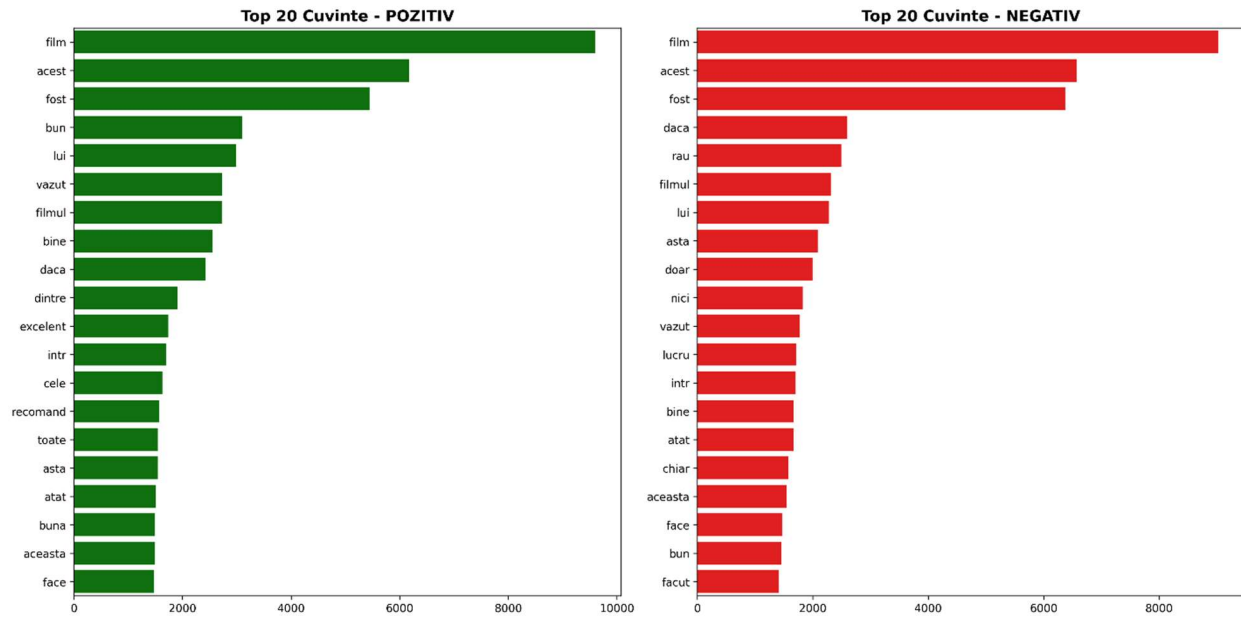
II. RO_SENT

Explorarea datelor:



Train este destul de dezechilibrat. Modelele pot tinde catre rezultatul pozitiv.





Se observa ca multe cuvinte sunt commune. Distributiile sunt similare.

Tokenizare si Embedding Layer:

- Curatarea datelor:
 - o Lowercase: uniformizare
 - o Eliminare taguri HTML
 - o Eliminare URL-uri
 - o Păstrare doar caractere alfabetice – punctuatia nu ajuta
 - o Normalizare spații multiple
- Vocabular maxim de 10.000 de cuvinte – acopera cuvintele relevante, elimina zgomot
- Frecventa minima de 2 aparitii – scoate typo-uri sau cuvinte car nu pot duce la generalizare
- Padding/cutoff pana la MAX_LEN = 95 percentile
- Unknown token
- Word2Vec cu vector de 100 dimensiuni
- Window = 5 cuvinte context

Arhitecturi:

Parametri comuni:

- Batch size : 64
- Epoci : 20

- Optimizator : Adam(lr = 0.001)
- Loss : Binary CrossEntropy
- Early stopping
- Learning Rate Scheduling

RNN:

- Embedding trainable = False : embedding-urile word2vec sunt deja bune si sunt mai putini parametri de antrenat
- SimpleRNN
- Dense(1, sigmoid) : output [0-1] interpreteaza ca probabilitate; threshold 0.5
- Rezultate:
 - o Test Accuracy: 57.12%
 - o Test Loss: 0.6848
 - o Early Stopping: Epoca 6 (patience atins rapid)
 - o ReduceLR: Epoca 4

RNN Regularized:

- Embedding -> SimpleRNN -> BatchNormalization -> Dense(62, relu) -> Dropout(0.5) -> Dense(1, sigmoid)
- Diferente fata de baseline:
 - Trainable = true : poate duce la overfitting dar e contrabalansat de dropout
 - SimpleRNN(128)
 - Dropout = 0.3 : previne memorarea
 - Recurrent_dropout = 0.2 : dropout pe secvente recurente
 - BatchNormalization
 - Dense(64, relu) + Dropout(0.5): strat conectat intermediar cu non-liniaritate; invata transformari non-liniare complexe si regularizeaza agresiv pe ultimul strat care e cel mai predisus la overfitting
- Rezultate:
 - Test Accuracy: 56.29%
 - Test Loss: 0.6828
 - Early Stopping: Epoca 7
 - ReduceLR: Epoca 5

LSTM Baseline:

- Embedding trainable = False : embeddings pre-antrenate fixe
- LSTM(64) : 64 = dimensiunea hidden state (memorie LSTM)
- Dense(1, sigmoid)
- Rezultate:
 - o Test Accuracy: 84.44%
 - o Test Loss: 0.3576
 - o Early Stopping: Epoca 14
 - o ReduceLR: : Epoca 12

LSTM Regularized:

- Embedding trainable = True -> LSTM(128, dropout =0.3, recurrent_dropout = 0.2) -> BatchNormalization -> Dense(64, relu, kernel_regularizer=l2(0.01)) -> Dropout(0.5) -> Dense(1, sigmoid)
- Imbunatatiri fata de LSTM Baseline:
 - o trainable=True + LSTM(128) : capacitate marita pentru pattern-uri complexe si fine-tuning embeddings pe sentiment
 - o kernel_regularizer = l2(0.01) : penalizeaza ponderile mari in Dense layer; forteaza ponderi mici pentru generalizare mai buna, previne overfitting
- Rezultate:
 - o Test Accuracy: 87.51%
 - o Test Loss: 0.3218
 - o Early Stopping: Epoca 13

BiLSTM:

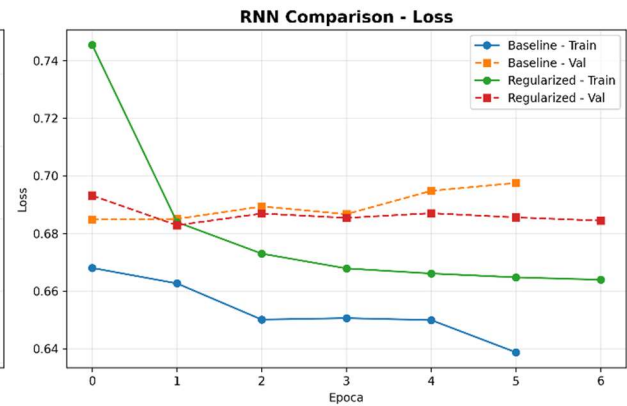
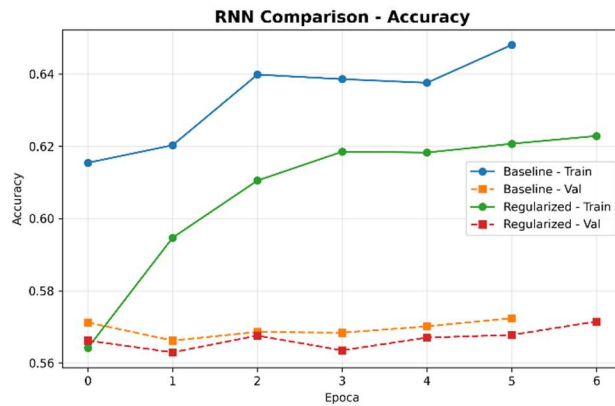
- Embedding trainable = True -> Bidirectional(LSTM(64, dropout=0.2, recurrent_dropout=0.2)) -> Dense(1, sigmoid)
- Rezultate:
 - o Test Accuracy: 86.06%
 - o Test Loss: 0.3420
 - o Early Stopping: Epoca 9
 - o ReduceLR: Epoca 7

Stacked BiLSTM:

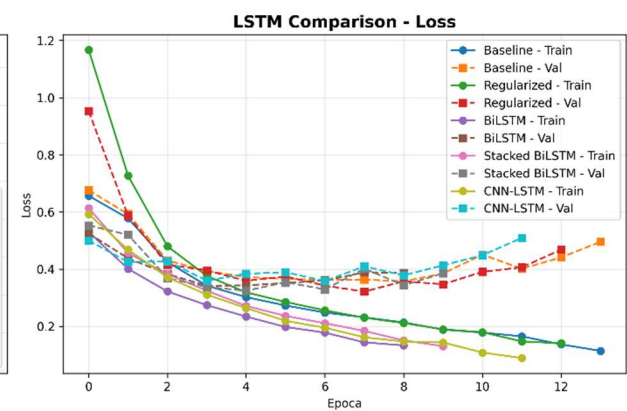
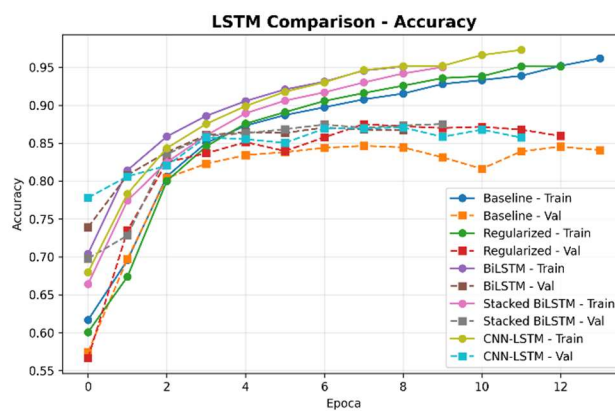
- Embedding trainable = True -> Bidirectional(LSTM(128, return_sequences=True, dropout=0.3, recurrent_dropout=0.2)) -> Bidirectional(LSTM(64, dropout=0.3, recurrent_dropout=0.2)) -> BatchNormalization -> Dense(64, relu) -> Dropout(0.5) -> Dense(1, sigmoid)
- Rezultate:
 - o Test Accuracy: 86.33%
 - o Test Loss: 0.3249
 - o Early Stopping: Epoca 10
 - o ReduceLR: Epoca 8

CNN-LSTM Hybrid:

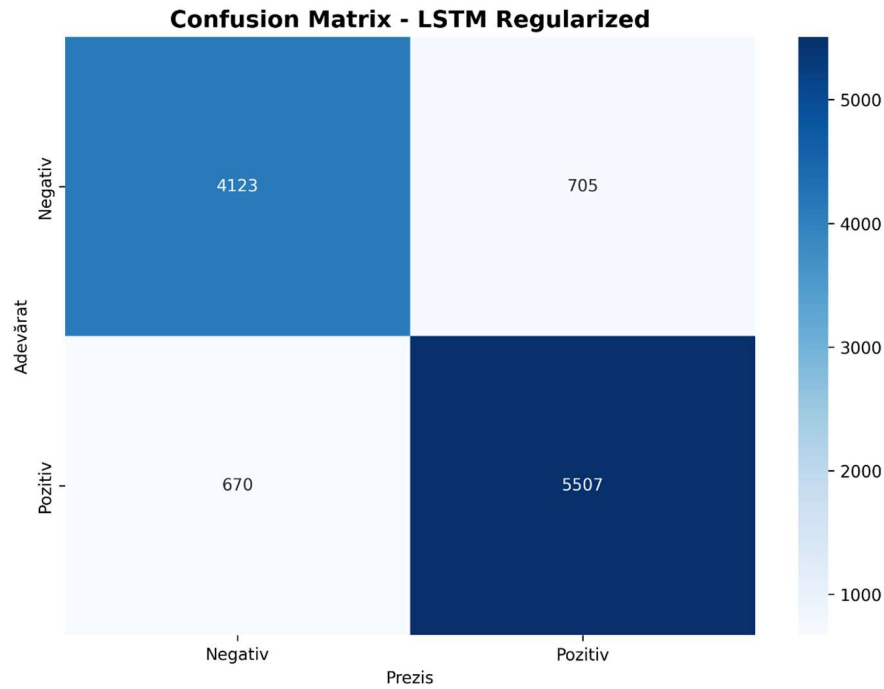
- Embedding trainable = True -> Conv1D(128, kernel_size=5, activation='relu') -> MaxPooling1D(pool_size=2) -> LSTM(64, dropout=0.2, recurrent_dropout=0.2) -> Dense(1, sigmoid)
- CNN vede pattern-uri locale, LSTM vede dependinte temporale, ordine si evolutie
- Rezultate:
 - o Test Accuracy: 86.92%
 - o Test Loss: 0.3590
 - o Early Stopping: Epoca 12



Se intampla overfitting destul de major. RNN Baseline a avut rezultate mai bune.



Mai puțin overfitting. Modelele sunt apropiate ca performanta.



	precision	recall	f1-score	support
Negativ	0.86	0.85	0.86	4828
Pozitiv	0.89	0.89	0.89	6177
accuracy			0.88	11005
macro avg	0.87	0.87	0.87	11005
weighted avg	0.87	0.88	0.88	11005

Model	Test Accuracy	Test Loss
LSTM Regularized	0.875057	0.321797
CNN-LSTM	0.869150	0.359014
Stacked BiLSTM	0.863335	0.324870
BiLSTM	0.860609	0.342000
LSTM Baseline	0.844434	0.357578
RNN Baseline	0.571195	0.684849
RNN Regularized	0.562926	0.682836

